



**Government of Karnataka
Department of Collegiate and Technical Education**

Government Polytechnic Turuvekere-155



**MINI PROJECT REPORT
ON
EMAIL SPAM DETECTION USING MACHINE LEARNING**

Submitted by

Prajwal M K

Computer Science and Engineering
Government polytechnic Turuvekere

Academic Year: 2026–2027

Submitted to:

PALLAVI B S

Course co-co-ordinator

Artificial Intelligence and Machine Learning.
Department of Computer Science and Engineering

1. Abstract

Email is widely used for personal, educational and business communication. However, unwanted or misleading messages known as spam emails can waste time and may contain fraudulent links or deceptive offers. This project demonstrates a basic machine-learning approach to classify email text into spam and non-spam categories.

The project is implemented in Python using a dataset stored in a CSV file. The data is inspected and cleaned, and the email text is converted into numerical features using Term Frequency–Inverse Document Frequency (TF-IDF). A Multinomial Naive Bayes classifier is trained on the processed data. The model is evaluated using accuracy, precision, recall, F1-score, a confusion matrix and a classification report.

The notebook also contains an interactive email checker. In its current version, this checker searches for dataset spam-email strings within the entered text. It is separate from the trained machine-learning model; therefore, its output should not be treated as a general prediction from the model. The report describes both the machine-learning workflow and this limitation.

2. Introduction

Spam email refers to unsolicited or unwanted electronic messages. Such messages may advertise products, promote suspicious offers or attempt to trick recipients into sharing information. Manually checking every message is inefficient, so automated text classification can help identify suspicious messages.

Machine learning allows a computer program to learn patterns from examples. In this project, labeled email messages are used to train a classifier. Each message has an associated label, such as spam or ham (non-spam). The trained model learns statistical relationships between words and labels and uses them to predict labels for unseen text.

3. Problem Statement

To develop and demonstrate a simple Python-based system that processes labeled email text and trains a machine-learning classifier to distinguish spam from non-spam messages, then evaluates its performance on a held-out test dataset.

4. Tools and Technologies

- Python: programming language used to implement the workflow.
- Jupyter Notebook: environment for writing and executing code interactively.
- Pandas: loading, inspecting and cleaning tabular data.
- NumPy: numerical computing support.
- Scikit-learn: data splitting, TF-IDF vectorization, model training and evaluation.

5. System Requirements

- Software: Python 3, Jupyter Notebook, Pandas, NumPy and Scikit-learn.
- Hardware: A standard computer capable of running Python and Jupyter Notebook.

6. Methodology

❖ Data Loading and Inspection

1. Load EMAIL.csv using Pandas.
2. Display a few rows of the dataset.
3. Check the number of rows and columns.
4. Check for missing values.
5. Check how many **Spam** and **Not Spam** emails are present.

❖ Data Preprocessing

1. Remove duplicate emails.
2. Remove rows with missing values.
3. Convert all email text to lowercase.
4. Select the **email** column as input X.
5. Select the **label** column as output y.

❖ Train-Test Split

1. Split the data into training and testing sets.
2. Use **75% for training**.

3. Use **25% for testing**.
4. Set `random_state=42`.
5. Use `stratify=y` to keep the same spam/not-spam ratio.

❖ **TF-IDF Feature Extraction**

1. Convert email words into numbers using **TF-IDF**.
2. Fit TF-IDF only on the training emails.
3. Transform the training emails into numerical features.
4. Transform the test emails using the same vectorizer.

❖ **Model Training**

1. Choose **Multinomial Naive Bayes**. - is a probabilistic machine learning algorithm specifically designed for classifying discrete data, most notably text corpora where features represent word counts or term frequencies
2. Give it the TF-IDF training data.
3. Train the model using the training labels.

❖ **Model Evaluation**

1. Use the trained model to predict the test emails.
2. Calculate:
 - **Accuracy** – overall correct predictions.
 - **Precision** – how many predicted spam emails were actually spam.
 - **Recall** – how many actual spam emails were detected.
 - **F1-score** – balance between precision and recall.
3. Display the **confusion matrix**.
4. Display the **classification report**.
5. Use these results to understand how well the model performs on unseen test emails.

6. Implementation and Execution

Follow these steps to run the project:

- Install Python and Jupyter Notebook.
- Install dependencies by running: `pip install pandas numpy scikit-learn jupyter`

- Keep EMAIL.csv and the notebook in the same folder.
- Open a terminal or command prompt in the project folder and run: jupyter notebook
- Open the spam detection notebook and execute the cells in order.
- Review the evaluation metrics and test the interactive checker.

10. Results and Discussion

The notebook produces accuracy, precision, recall, F1-score, a confusion matrix and a classification report after model evaluation. Insert the actual values printed by your notebook in the table below after running all cells.

Evaluation metric	Value from notebook
Accuracy (%)	80.0 %
Precision (spam)	0.67
Recall (spam)	1.0
F1-score (spam)	0.8
Confusion matrix	[2 1] [0 2]

11. Conclusion

This project demonstrates the basic steps of an email text-classification workflow: data loading, cleaning, train-test splitting, TF-IDF feature extraction, Naive Bayes training and performance evaluation. It provides practical experience with Python, Pandas and Scikit-learn.

The current interactive checker uses dataset string matching and should be distinguished from the trained classifier. Connecting the checker to the fitted TF-IDF vectorizer and Naive Bayes model is an important improvement for classifying messages that were not present verbatim in the dataset.

12. References

- Scikit-learn documentation: <https://scikit-learn.org/>
- Pandas documentation: <https://pandas.pydata.org/docs/>
- Python documentation: <https://docs.python.org/3/>



```
In [1]: #importing libararies
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    precision_score,
    recall_score,
    f1_score
)
```

```
In [2]: #data handling
df = pd.read_csv('EMAIL.csv')

print("Dataset:")
print(df.head())

print("\nDataset Shape:")
print(df.shape)

print("\nMissing Values:")
print(df.isnull().sum())

print("\nClass Distribution: \n")
print(df["label"].value_counts())
```

```
Dataset:
          email label
0  Congratulations you won a free prize  spam
1          Claim your cash reward now  spam
2      Win money by clicking this link  spam
3          You have won a lottery  spam
4      Free gift waiting for you  spam
```

```
Dataset Shape:
(20, 2)
```

```
Missing Values:
email    0
label    0
dtype: int64
```

```
Class Distribution:

spam    10
ham     10
Name: label, dtype: int64
```

```
In [3]: #dropping duplicate values
df = df.drop_duplicates()

# Remove missing values
df = df.dropna()

# Convert text to lowercase
df["email"] = df["email"].str.lower()

# Separate input and output
X = df["email"]
y = df["label"]
```

```
In [5]: #data splitting
X_train, X_test, y_train, y_test = train_test_split( X, y,
    test_size=0.25,
    random_state=42,
    stratify=y
)
#data size
print("\nTraining data:", len(X_train))
print("Testing data:", len(X_test))
```

Training data: 15
Testing data: 5

```
In [6]: #Convert text into numerical features
vectorizer = TfidfVectorizer()

X_train_tfidf = vectorizer.fit_transform(X_train)

X_test_tfidf = vectorizer.transform(X_test)

print("\nTF-IDF Training Shape:")
print(X_train_tfidf.shape)
```

TF-IDF Training Shape:
(15, 55)

```
In [8]: #model selection and training
model = MultinomialNB()

model.fit(X_train_tfidf, y_train)

print("\nModel Training Completed!")
```

Model Training Completed!

```
In [9]: y_pred=model.predict(X_test_tfidf)
```

```
In [10]: accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(
    y_test, y_pred, pos_label="spam",
```

```

    zero_division=0
)

recall = recall_score(
    y_test, y_pred, pos_label="spam",
    zero_division=0
)

f1 = f1_score(
    y_test, y_pred, pos_label="spam",
    zero_division=0
)

#print("\n===== MODEL EVALUATION =====")

print("Accuracy:", round(accuracy * 100, 2), "%")
print("Precision:", round(precision, 2))
print("Recall:", round(recall, 2))
print("F1 Score:", round(f1, 2))

print("\nConfusion Matrix:")
print(confusion_matrix(
    y_test, y_pred,
    labels=["ham", "spam"]
))

print("\nClassification Report:")
print(classification_report(
    y_test, y_pred,
    zero_division=0
))

```

Accuracy: 80.0 %
 Precision: 0.67
 Recall: 1.0
 F1 Score: 0.8

Confusion Matrix:
 [[2 1]
 [0 2]]

Classification Report:

	precision	recall	f1-score	support
ham	1.00	0.67	0.80	3
spam	0.67	1.00	0.80	2
accuracy			0.80	5
macro avg	0.83	0.83	0.80	5
weighted avg	0.87	0.80	0.80	5

In [20...

```

print("\n===== EMAIL SPAM CHECKER =====")

spam_sender = [

```

```

    'fake@gmail.com',
    'scammer@gmail.com'
]

spam_email = []
not_spam_email = []

# Common spam words
spam_words = [
    'win',
    'winner',
    'free',
    'prize',
    'offer',
    'urgent',
    'money',
    'click here',
    'lottery',
    'congratulations'
]

# Get spam emails from the dataset
spam_dataset = df[
    df['label'].str.lower() == 'spam'
]['email'].str.lower().tolist()

# Combine dataset spam emails and spam words
spam_check = spam_dataset + spam_words

while True:

    sender = input(
        "\nEnter the sender address or type 'exit': "
    ).lower().strip()

    # Exit program
    if sender == 'exit':
        print("\nExiting Spam Checker...")
        print("Thank you for using the Spam Checker!")
        break

    # Check sender address
    if sender in spam_sender:

        print("\nEmail Status = SPAM")
        print("Sender:", sender)

        spam_email.append({
            'sender': sender,
            'email': 'Sender found in spam list',
            'status': 'SPAM'
        })

```

```

else:

    # Get email message
    email = input(
        "\nEnter email message or type 'exit': "
    ).lower().strip()

    # Exit program
    if email == 'exit':
        print("\nExiting Spam Checker...")
        print("Thank you for using the Spam Checker!")
        break

    # Check email against spam dataset and spam words
    if any(word in email for word in spam_check):

        print("\nEmail Status = SPAM")

        spam_email.append({
            'sender': sender,
            'email': email,
            'status': 'SPAM'
        })

    else:

        print("\nEmail Status = NOT SPAM")

        not_spam_email.append({
            'sender': sender,
            'email': email,
            'status': 'NOT SPAM'
        })

# =====
# CREATE DATAFRAMES
# =====

spam_df = pd.DataFrame(spam_email)
not_spam_df = pd.DataFrame(not_spam_email)

# Display Spam emails
print("\n===== SPAM EMAILS =====")
print(spam_df)

# Display Not Spam emails
print("\n===== NOT SPAM EMAILS =====")
print(not_spam_df)

# =====
# COMBINE BOTH DATAFRAMES
# =====

```

```

all_email_df = pd.concat(
    [spam_df, not_spam_df],
    ignore_index=True
)

print("\n===== ALL EMAIL DETAILS =====")
print(all_email_df)

# Save results to CSV
all_email_df.to_csv(
    "email_spam_results.csv",
    index=False
)

print("\nResults saved to email_spam_results.csv")

```

===== EMAIL SPAM CHECKER =====

Email Status = SPAM
 Sender: fake@gmail.com

Email Status = NOT SPAM

Exiting Spam Checker...
 Thank you for using the Spam Checker!

===== SPAM EMAILS =====

	sender	email status
0	fake@gmail.com	Sender found in spam list SPAM

===== NOT SPAM EMAILS =====

	sender email	status
0	prajwal45@gmil.com hlo	NOT SPAM

===== ALL EMAIL DETAILS =====

	sender	email	status
0	fake@gmail.com	Sender found in spam list	SPAM
1	prajwal45@gmil.com	hlo	NOT SPAM

Results saved to email_spam_results.csv

In []:

In []: